

**School of Computer Science and Artificial Intelligence**

---

**Lab Assignment # 16.2**

---

**Program** : B. Tech (CSE)  
**Specialization** : -  
**Course Title** : AI Assisted Coding  
**Course Code** : 23CS002PC304  
**Semester** II  
**Academic Session** : 2025-2026  
**Name of Student** : K. Deepak  
**Enrollment No.** : 2303A51T02  
**Batch No** : 52  
**Date** : 17/03/26

---

**Task 1: (Design Database Schema for Student Management System )**

**Input:**

Schema SQL •

```
1 CREATE TABLE Students (  
2     student_id INT PRIMARY KEY,  
3     name VARCHAR(100),  
4     email VARCHAR(100)  
5 );  
6  
7 CREATE TABLE Courses (  
8     course_id INT PRIMARY KEY,  
9     course_name VARCHAR(100),  
10    credits INT  
11 );  
12  
13 CREATE TABLE Enrollments (  
14     enrollment_id INT PRIMARY KEY,  
15     student_id INT,  
16     course_id INT,  
17     FOREIGN KEY (student_id) REFERENCES Students(student_id),  
18     FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
19 );  
20  
21 -- SHOW OUTPUT  
22 SHOW TABLES;  
23 DESCRIBE Students;  
24 DESCRIBE Courses;  
25 DESCRIBE Enrollments;
```

## Output:

Results Copy as Markdown

Query #5 Execution time: 0.33ms

| Field      | Type         | Null | Key | Default | Extra |
|------------|--------------|------|-----|---------|-------|
| student_id | int(11)      | NO   | PRI | null    |       |
| name       | varchar(100) | YES  |     | null    |       |
| email      | varchar(100) | YES  |     | null    |       |

Query #6 Execution time: 0.32ms

| Field       | Type         | Null | Key | Default | Extra |
|-------------|--------------|------|-----|---------|-------|
| course_id   | int(11)      | NO   | PRI | null    |       |
| course_name | varchar(100) | YES  |     | null    |       |
| credits     | int(11)      | YES  |     | null    |       |

Query #7 Execution time: 0.37ms

| Field         | Type    | Null | Key | Default | Extra |
|---------------|---------|------|-----|---------|-------|
| enrollment_id | int(11) | NO   | PRI | null    |       |
| student_id    | int(11) | YES  | MUL | null    |       |
| course_id     | int(11) | YES  | MUL | null    |       |

## Explanation:

1. Defines structure of database using tables: Students, Courses, Enrollments.
2. Primary keys ensure unique identification of each record.
3. Foreign keys maintain referential integrity between related tables.
4. Follows normalization (1NF, 2NF, 3NF) to remove redundancy and ensure consistency.

## Task 2: (Insert Sample Records)

### Input:

 Database: MySQL v5.7
 Run
Update
Fork
Load Example
Star
PRO

Query SQL PRO
Have any feedback?

```

1 INSERT INTO Students VALUES
2 (1, 'Arjun', 'arjun@gmail.com'),
3 (2, 'Meera', 'meera@gmail.com'),
4 (3, 'Rahul', 'rahul@gmail.com');
5
6 INSERT INTO Courses VALUES
7 (101, 'Database Systems', 4),
8 (102, 'Data Structures', 3),
9 (103, 'Operating Systems', 4),
10 (104, 'AI Fundamentals', 3);
11
12 INSERT INTO Enrollments VALUES
13 (1, 1, 101),
14 (2, 1, 102),
15 (3, 2, 103),
16 (4, 3, 101),
17 (5, 3, 104);
18
19 -- ☒ SHOW RESULT
20 SELECT * FROM Students;
21 SELECT * FROM Courses;
22 SELECT * FROM Enrollments;
    
```

### Output:

Results
 Copy as Markdown

| student_id | name  | email           |
|------------|-------|-----------------|
| 1          | Arjun | arjun@gmail.com |
| 2          | Meera | meera@gmail.com |
| 3          | Rahul | rahul@gmail.com |

Query #5 Execution time: 0.07ms

| course_id | course_name       | credits |
|-----------|-------------------|---------|
| 101       | Database Systems  | 4       |
| 102       | Data Structures   | 3       |
| 103       | Operating Systems | 4       |
| 104       | AI Fundamentals   | 3       |

Query #6 Execution time: 0.06ms

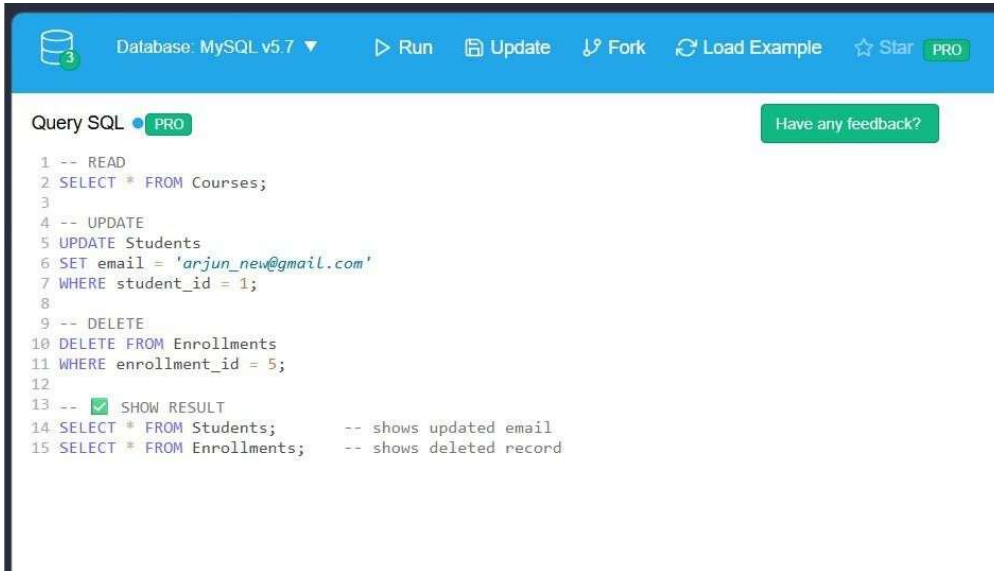
| enrollment_id | student_id | course_id |
|---------------|------------|-----------|
| 1             | 1          | 101       |
| 2             | 1          | 102       |
| 3             | 2          | 103       |
| 4             | 3          | 101       |
| 5             | 3          | 104       |

### Explanation:

1. Populates tables with initial data using INSERT statements.
2. Ensures relational links by matching student\_id and course\_id.
3. Helps in testing database functionality with sample dataset.
4. SELECT queries verify whether data insertion is successful.

## Task 3: CRUD Operations

### Input:



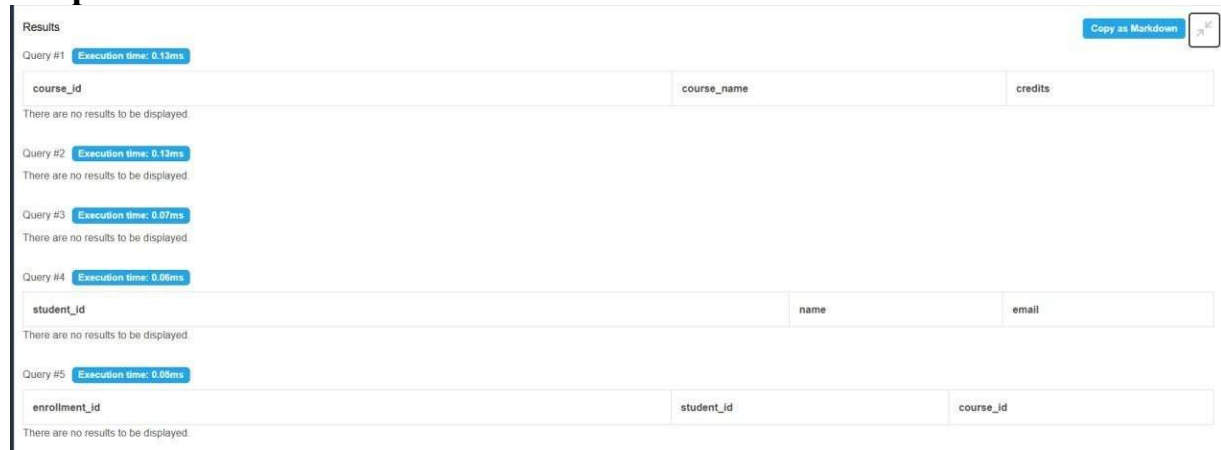
The screenshot shows a MySQL query editor with a blue header bar containing icons for database operations and a 'Database: MySQL v5.7' dropdown. Below the header, the query text is as follows:

```

1 -- READ
2 SELECT * FROM Courses;
3
4 -- UPDATE
5 UPDATE Students
6 SET email = 'arjun_new@gmail.com'
7 WHERE student_id = 1;
8
9 -- DELETE
10 DELETE FROM Enrollments
11 WHERE enrollment_id = 5;
12
13 -- ☒ SHOW RESULT
14 SELECT * FROM Students;      -- shows updated email
15 SELECT * FROM Enrollments;  -- shows deleted record
  
```

A green 'PRO' badge is visible next to the 'Query SQL' label, and a 'Have any feedback?' button is on the right.

### Output:



The screenshot displays the 'Results' section of the MySQL interface. It contains five query results, each with a table header and a message stating 'There are no results to be displayed.'

| course_id                             | course_name | credits |
|---------------------------------------|-------------|---------|
| There are no results to be displayed. |             |         |

| student_id                            | name | email |
|---------------------------------------|------|-------|
| There are no results to be displayed. |      |       |

| enrollment_id                         | student_id | course_id |
|---------------------------------------|------------|-----------|
| There are no results to be displayed. |            |           |

### Explanation:

1. SELECT retrieves data from tables for viewing records.
2. UPDATE modifies existing data like student email.
3. DELETE removes specific records such as enrollments.
4. Validates operations using SELECT to ensure correctness.

## Task 4: Aggregate Queries

### Input:

Database: MySQL v5.7

Run Update Fork Load Example Star PRO

Query SQL PRO

Query successfully executed in 50.44ms

```

1 -- Count students per course
2 SELECT c.course_name, COUNT(e.student_id) AS student_count
3 FROM Courses c
4 JOIN Enrollments e ON c.course_id = e.course_id
5 GROUP BY c.course_name;
6
7 -- Courses with more than 1 student
8 SELECT c.course_name
9 FROM Courses c
10 JOIN Enrollments e ON c.course_id = e.course_id
11 GROUP BY c.course_name
12 HAVING COUNT(e.student_id) > 1;
  
```

### Output:

Results PRO

Copy as Markdown

Query #1 Execution time: 0.2ms

| course_name                           | student_count |
|---------------------------------------|---------------|
| There are no results to be displayed. |               |

Query #2 Execution time: 0.16ms

| course_name                           |
|---------------------------------------|
| There are no results to be displayed. |

### Explanation:

1. Uses functions like COUNT() to summarize data.
2. GROUP BY groups records based on course names.
3. HAVING filters grouped data (e.g., courses with >1 student).
4. Helps in analysis like enrollment trends per course.

## Task 5: Query Optimization

### Input:


Database: MySQL v5.7 ▼
Run
Update
Fork
Load Example
Star
PRO

Query SQL
PRO

Query successfully executed in 8.19ms

```

1 -- Original
2 SELECT * FROM Students
3 WHERE student_id IN (
4     SELECT student_id FROM Enrollments WHERE course_id = 101
5 );
6
7 -- Optimized
8 SELECT s.*
9 FROM Students s
10 JOIN Enrollments e ON s.student_id = e.student_id
11 WHERE e.course_id = 101;
    
```

### Output:


Database: MySQL v5.7 ▼
Run
Update
Fork
Load Example
Star
PRO

Results
PRO
Copy as Markdown

Query #1 Execution time: 0.25ms

| student_id                            | name | email |
|---------------------------------------|------|-------|
| There are no results to be displayed. |      |       |

Query #2 Execution time: 0.14ms

| student_id                            | name | email |
|---------------------------------------|------|-------|
| There are no results to be displayed. |      |       |

### Explanation:

1. Identifies inefficient queries (e.g., subqueries).
2. Rewrites queries using JOINS for better performance.
3. Ensures both original and optimized queries give same result.
4. Improves execution speed, especially for large datasets.